

-Họ và tên: Phạm Minh Đạt

-MSSV: 20210169

-Module phụ trách: Subsystem by vietQR (Payment Subsystem - VietQR)

-Mã nguồn phân tích thiết kế sẽ là: VietQRSubsystemController.java và PaymentForm.java

1. Đánh giá Cohesion (Độ dính kết)

Module được chọn để đánh giá: Lớp VietQRSubsystemController (Đây là lớp chịu trách nhiệm xử lý logic sinh mã QR)

a. Liệt kê các thành phần:

- Thuộc tính (Attributes):

1. BANK_ID (Mã ngân hàng)
2. ACCOUNT_NO (Số tài khoản)
3. ACCOUNT_NAME (Tên chủ tài khoản)
4. TEMPLATE (Mẫu giao diện QR)

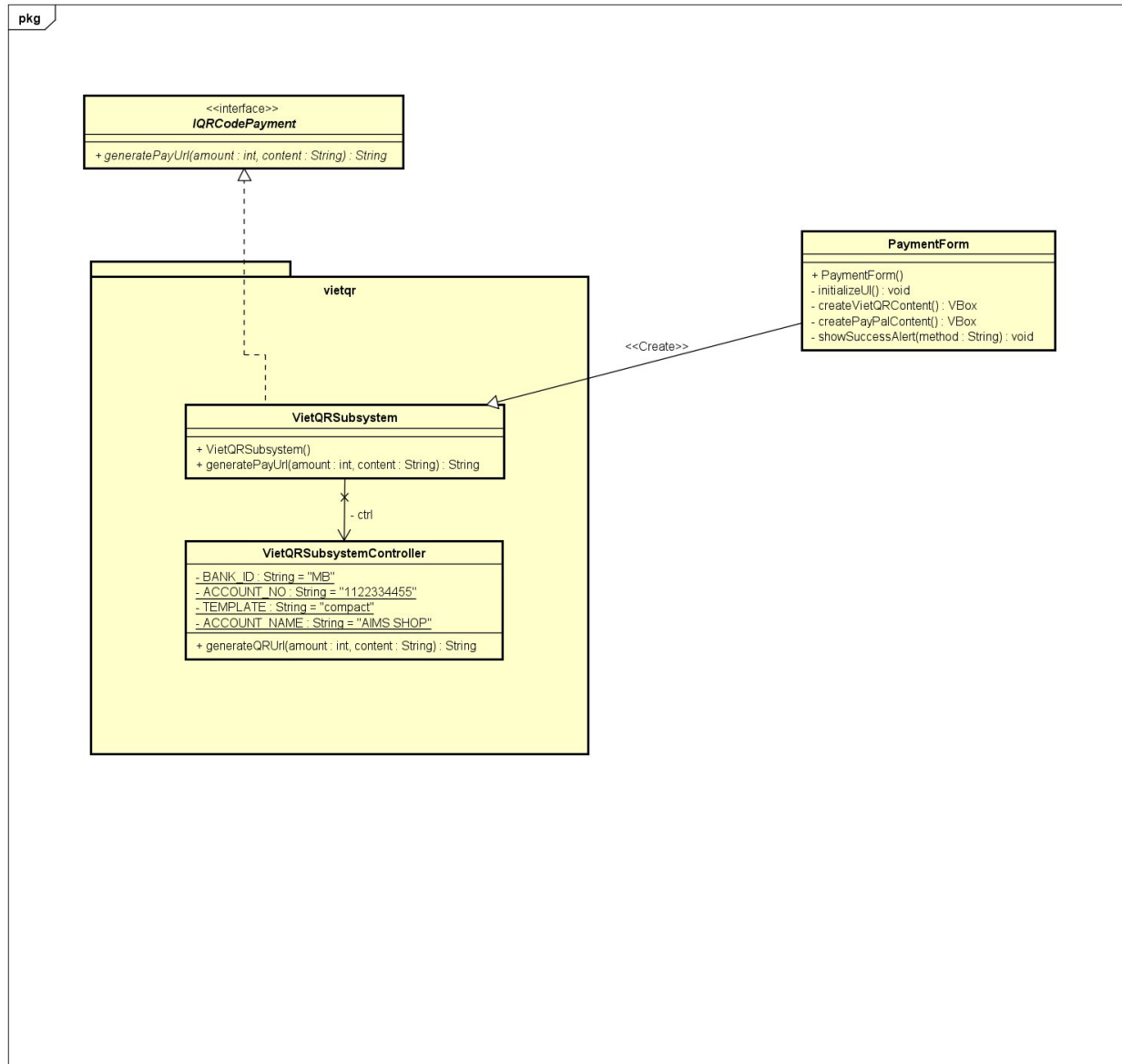
- Phương thức (Methods):

1. generateQRUrl(int amount, String content)

b. Phân tích sự liên quan (Graph): Trong phương thức generateQRUrl, chúng ta thấy:

- Để tạo ra chuỗi URL, phương thức này **sử dụng tất cả 4 thuộc tính** (BANK_ID, ACCOUNT_NO, ACCOUNT_NAME, TEMPLATE) kết hợp với tham số đầu vào (amount, content).
- Không có thuộc tính nào bị thừa, không có phương thức nào làm việc không liên quan đến việc tạo QR.

Sơ đồ liên kết :



c. Kết luận:

- **Mức độ Cohesion: Functional Cohesion** (Mức cao nhất - Tốt nhất).
- **Lý do:** Mọi thành phần trong lớp đều cùng phục vụ một mục đích chức năng duy nhất là "Tạo chuỗi URL thanh toán VietQR". Lớp này không ôm đồm các việc khác (như gửi email hay lưu database).
- **Đánh giá:** Thiết kế đảm bảo High Cohesion.

2. Đánh giá Coupling (Độ phụ thuộc)

Mối quan hệ được đánh giá: Giữa lớp giao diện `PaymentForm` và lớp xử lý `VietQRSubsystem`.

a. Phân tích mã nguồn: Trong `PaymentForm`, chúng ta gọi `Subsystem` như sau:

```
// 1. Khởi tạo Subsystem thông qua Interface
IQRCodePayment paymentSubsystem = new VietQRSubsystem();

// 2. Gọi hàm generatePayUrl (Số tiền giả định 150k)
String qrUrl = paymentSubsystem.generatePayUrl( amount: 150000, content: "Thanh toán đơn hàng AIMS");
```

b. Đánh giá mức độ Coupling:

1. **Sử dụng Interface:** `PaymentForm` giao tiếp thông qua Interface `IQRCodePayment`.
 - Điều này giúp giảm sự phụ thuộc vào cài đặt cụ thể. Nếu sau này đổi logic bên trong `VietQRSubsystem`, code ở `PaymentForm` không bị ảnh hưởng (miễn là Interface không đổi).
2. **Truyền dữ liệu:** Dữ liệu truyền đi là các kiểu dữ liệu nguyên thủy (`int amount`, `String content`).
 - Đây là **Data Coupling** (Mức độ phụ thuộc thấp/tốt). Chúng ta không truyền cả một object khổng lồ (như `Order` hay `Cart`) vào `Subsystem`, giúp tránh việc `Subsystem` biết quá nhiều về cấu trúc dữ liệu của App.

c. Kết luận:

- **Mức độ Coupling:** **Data Coupling** (Tốt).
- **Đánh giá:** Thiết kế hiện tại đảm bảo Low Coupling nhờ áp dụng mẫu thiết kế **Facade** (thông qua `Subsystem`) và sử dụng **Interface**.

d. Đề xuất cải tiến (Để bài báo cáo sâu sắc hơn): Mặc dù coupling đã thấp, nhưng trong `PaymentForm` vẫn còn dòng lệnh khởi tạo trực tiếp: `new VietQRSubsystem()` (Vẫn còn chút phụ thuộc vào tên lớp cụ thể).

- **Giải pháp đề xuất:** Sử dụng **Abstract Factory Pattern** hoặc **Dependency Injection**.
- **Cách làm:** Thay vì `new` trực tiếp trong Form, chúng ta sẽ nhờ một Factory tạo đối tượng này.
 - Ví dụ: `IQRCodePayment payment = PaymentFactory.getPaymentSystem("VietQR");`
 - Khi đó `PaymentForm` sẽ hoàn toàn không biết đến sự tồn tại của class `VietQRSubsystem`, đạt mức độ lỏng lẻo tuyệt đối.